

TRABAJO DE FIN DE GRADO

Orchestration Trade-offs in Edge Computing: Systemd vs Kubernetes

Análisis de rendimiento, distribución de cargas e inferencia de modelos de IA en el extremo de la red

Autor:

[Tu Nombre y Apellidos]

Director/es:

[Nombre del Director/a]

Grado en Ingeniería [Informática / Telecomunicaciones]

[Nombre de tu Universidad]

[Mes y Año de la convocatoria]

Índice

1	Introducción	1
1.1	Contexto y Motivación	1
1.2	Objetivos del Trabajo	1
1.3	Estructura de la Memoria	2
2	Estado del Arte y Tecnologías Involucradas	3
2.1	Paradigmas de Computación: Cloud vs. Edge Computing	3
2.2	Tecnologías de Contenedorización Ligera	3
2.3	Orquestación y Ciclo de Vida en el Edge	3
2.3.1	Systemd y la evolución hacia Quadlets	3
2.3.2	Distribuciones Ligeras de Kubernetes (K3s)	3
2.4	Protocolos de Comunicación y Serialización para Alta Frecuencia	4
2.4.1	Arquitecturas HTTP/REST y RPC (gRPC)	4
2.4.2	Mensajería Asíncrona y Punto a Punto (ZeroMQ, MQTT)	4
2.4.3	Estrategias de Serialización: JSON vs Protobuf vs MessagePack	4
2.5	Machine Learning Distribuido para Inferencia	4
3	Metodología y Evolución de la Arquitectura	5
3.1	Enfoque de Desarrollo Iterativo	5
3.2	Infraestructura Física y Virtual	5
3.3	Aprovisionamiento y Despliegue Agnóstico	5
3.4	Casos de Uso y Definición de Tareas	6
3.4.1	Tarea 1: Transmisión masiva de datos (Conteo de imágenes)	6
3.4.2	Tarea 2: Inferencia distribuida de Inteligencia Artificial	6
3.5	Arquitectura Lógica de la Solución	6
4	Fase 1: Transmisión de Datos en Systemd (Conteo de Imágenes)	7
4.1	Definición de la Tarea	7
4.2	Despliegue de la Infraestructura mediante Systemd Quadlets	7
4.3	Diseño del Banco de Pruebas de Comunicaciones	7
4.4	Diseño del Experimento (Comparativa)	7
4.5	Resultados y Análisis de Rendimiento	8
4.6	Resultados: Impacto de la Serialización	8
4.7	Conclusiones de la Fase 1	8
5	Fase 2: Despliegue e Inferencia de Modelos de IA	9
5.1	Entrenamiento del Modelo en el Master	9
5.2	Distribución del Modelo y Datos	9
5.3	Ejecución de Inferencia en el Edge (Workers)	9
5.4	Métricas de Procesamiento (T_{proc})	10

6	Fase 3: Migración a Kubernetes (K3s) y Análisis de Trade-offs	11
6.1	Migración de la Infraestructura a K3s	11
6.2	Ejecución de Cargas de Trabajo (Fase 1 y 2 en K8s)	11
6.3	Comparativa Directa: Systemd vs Kubernetes	11
7	Conclusiones y Trabajo Futuro	13
7.1	Conclusiones Finales	13
7.2	Respuesta al Dilema: ¿Systemd o Kubernetes para el Edge?	13
7.3	Trabajo Futuro	13

Índice de figuras

3.1 Topología del sistema y flujos de comunicación.	6
---	---

Índice de cuadros

4.1	Resumen de resultados de transmisión para 60,000 imágenes.	8
-----	--	---

Capítulo 1

Introducción

Este capítulo debe introducir al lector al problema, por qué es importante y qué vas a resolver.

1.1 Contexto y Motivación

Aquí debes hablar sobre el auge del *Edge Computing* (computación en el extremo) y cómo difiere del *Cloud Computing* tradicional. Explica que los entornos Edge tienen recursos limitados (CPU, RAM, red inestable) y que las herramientas de orquestación diseñadas para la nube (como Kubernetes) pueden ser demasiado pesadas. Introduce el dilema: ¿Necesitamos orquestadores complejos o bastan herramientas nativas del sistema operativo como Systemd + Podman?

”El auge del *Edge Computing* plantea nuevos retos en la orquestación de servicios descentralizados. Los entornos perimetrales poseen recursos limitados que cuestionan la viabilidad de modelos *cloud-native* puros.”

1.2 Objetivos del Trabajo

- **Objetivo Principal:** Analizar y comparar los *trade-offs* (rendimiento, resiliencia y complejidad) entre Kubernetes y Systemd como orquestadores para cargas de trabajo Edge.
- **Objetivos Específicos:**
 - Desplegar una infraestructura Master-Worker utilizando Ansible, Podman y Systemd (Fase 1).
 - Evaluar el rendimiento de múltiples protocolos de red (HTTP, gRPC, ZeroMQ, MQTT) y formatos de serialización (JSON, Protobuf, MessagePack) para la transmisión masiva de datos (Dataset MNIST).

- Implementar un flujo de trabajo de Machine Learning distribuido (Fase 2) entrenando un modelo PyTorch en el nodo máster e infiriendo en los nodos worker.
- Migrar la arquitectura a un entorno de Kubernetes/K3s (Fase 3) y contrastar las métricas de consumo y latencia frente a Systemd.

”El objetivo principal es analizar y comparar los *trade-offs* (rendimiento, consumo y complejidad) entre una orquestación basada en Systemd (nativa y ligera) frente a Kubernetes (robusta pero pesada) en escenarios Edge.”

1.3 Estructura de la Memoria

Breve resumen de lo que contiene cada uno de los siguientes capítulos.

”El documento se estructura siguiendo la progresión del desarrollo técnico iterativo, desde la base teórica hasta la comparativa final.”

Capítulo 2

Estado del Arte y Tecnologías Involucradas

Define teóricamente las herramientas y conceptos que usas. No es un manual de uso, sino una justificación técnica.

”Este capítulo compila la investigación técnica previa (correspondiente a las *Setmanas 4* y *5*) necesaria para fundamentar el diseño arquitectónico del proyecto.”

2.1 Paradigmas de Computación: Cloud vs. Edge Computing

Definición de conceptos y necesidades de orquestación en el extremo.

Desafíos en el Edge (huella de memoria, latencia, ancho de banda).

2.2 Tecnologías de Contenedorización Ligera

Análisis de Podman frente a Docker, destacando su arquitectura *daemonless* y ejecución *rootless*, ideales para entornos perimetrales seguros y con pocos recursos.

2.3 Orquestación y Ciclo de Vida en el Edge

2.3.1 Systemd y la evolución hacia Quadlets

Gestión del ciclo de vida de los contenedores de forma nativa desde el sistema operativo.

2.3.2 Distribuciones Ligeras de Kubernetes (K3s)

Concepto de K3s como estándar de facto para llevar la semántica de Kubernetes al Edge.

2.4 Protocolos de Comunicación y Serialización para Alta Frecuencia

2.4.1 Arquitecturas HTTP/REST y RPC (gRPC)

2.4.2 Mensajería Asíncrona y Punto a Punto (ZeroMQ, MQTT)

2.4.3 Estrategias de Serialización: JSON vs Protobuf vs MessagePack

Comparativa teórica entre texto plano con alto *overhead* y esquemas binarios eficientes.

- **REST / HTTP(s) y JSON:** Estándar web, basado en texto. Cuellos de botella en el parseo.
- **gRPC y Protocol Buffers:** Llamadas a procedimientos remotos sobre HTTP/2. Serialización binaria estructurada y fuertemente tipada.
- **ZeroMQ:** Sockets TCP crudos sin *broker* (Brokerless P2P). Alta velocidad.
- **MQTT:** Patrón Pub/Sub con *broker* intermediario.
- **MessagePack:** Serialización binaria sin esquema (*schema-less*), flexible y rápida.

2.5 Machine Learning Distribuido para Inferencia

Breve introducción a PyTorch, inferencia de modelos, y el dataset MNIST.

Introducción a PyTorch y las características del dataset MNIST utilizado para las cargas de trabajo.

Capítulo 3

Metodología y Evolución de la Arquitectura

3.1 Enfoque de Desarrollo Iterativo

El proyecto se ha abordado mediante iteraciones incrementales semanales. Se partió del aprovisionamiento base (*Setmana 0 a 3*), pasando por la optimización de protocolos (*Setmana 4*), la integración de IA (*Setmana 5*) y culminando en la migración a Kubernetes (*Setmana 6*).

3.2 Infraestructura Física y Virtual

Detalla el hardware:

- Nodo Master (Padre): Ubuntu 22.04.5 bajo WSL en Windows 11. Rol: Orquestador (Ansible), entrenamiento de IA, y despachador de tareas.
- Nodos Workers (Hijos): 2 Máquinas Virtuales Ubuntu 24.04.4 en VMware. Rol: Inferencia, cálculo, retorno de métricas.

3.3 Aprovisionamiento y Despliegue Agnóstico

Explica la separación estricta entre *Build* (construcción en el máster) y *Run* (ejecución en el worker). Detalla cómo Ansible transporta los artefactos locales (‘.tar’) usando ‘podman load’ (estrategia *side-loading*) para ahorrar ancho de banda y CPU, evitando repositorios externos (Docker Hub).

3.4 Casos de Uso y Definición de Tareas

3.4.1 Tarea 1: Transmisión masiva de datos (Conteo de imágenes)

División del dataset en N particiones y retorno del conteo.

3.4.2 Tarea 2: Inferencia distribuida de Inteligencia Artificial

Sustitución del conteo simple por la predicción numérica usando modelos pre-entrenados.

3.5 Arquitectura Lógica de la Solución

Aquí debes poner el esquema de la arquitectura Master-Worker, mostrando cómo fluyen los datos (Fase 2, Fase 3, Fase 4).

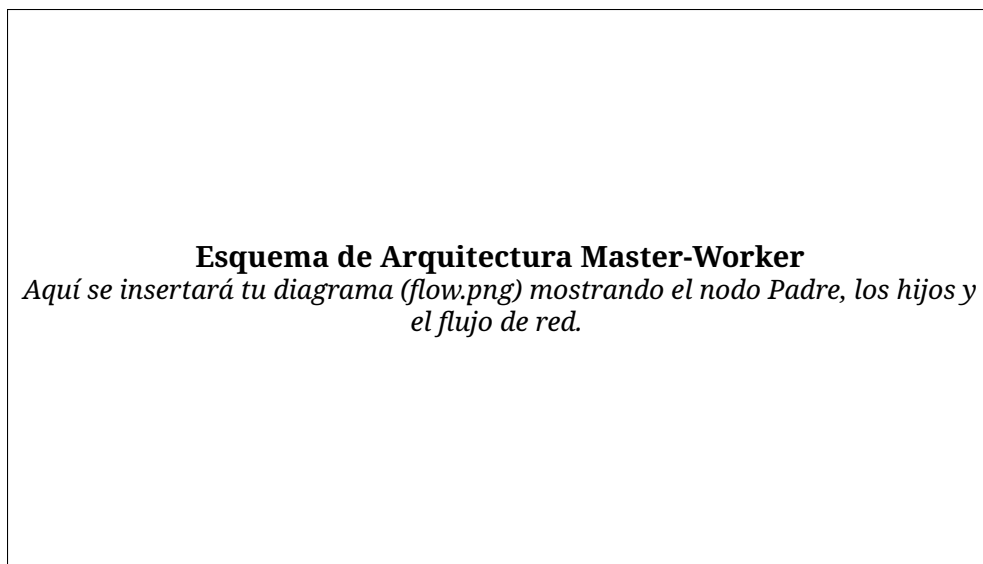


Figura 3.1: Topología del sistema y flujos de comunicación.

Capítulo 4

Fase 1: Transmisión de Datos en Systemd (Conteo de Imágenes)

En este capítulo desarrollas todo lo que me has pasado sobre el conteo de imágenes y la comparativa de red.

4.1 Definición de la Tarea

El Master descarga el dataset MNIST (60,000 imágenes), lo carga en memoria, lo divide en N particiones ($tamaño_particion = total_imagenes / N_nodos$) y envía los lotes. El Worker cuenta las imágenes recibidas y devuelve el total al Master.

4.2 Despliegue de la Infraestructura mediante Systemd Quadlets

Explicación de cómo se estructuraron los servicios base de forma agnóstica a la carga de trabajo.

4.3 Diseño del Banco de Pruebas de Comunicaciones

Metodología para medir latencias y consumos enviando 60.000 imágenes a los nodos.

4.4 Diseño del Experimento (Comparativa)

Explica la justificación de cada prueba:

- **Línea Base:** HTTP/1.1 + JSON (Demostrar ineficiencia del texto plano).
- **Estándar Microservicios:** gRPC + Protobuf (HTTP/2 binario).
- **Rendimiento Crudo:** ZeroMQ + Protobuf (Límite físico P2P).

- **IoT Broker:** MQTT + Protobuf (Penalización por doble salto).
- **Flexibilidad:** ZeroMQ + MessagePack (Esquema estático vs dinámico).

4.5 Resultados y Análisis de Rendimiento

Inserta aquí las tablas y gráficas de la Fase 1. Comenta el consumo extremo de RAM del JSON frente al paradigma binario (*Zero-Copy* con `np.frombuffer`).

Protocolo	Formato	Tiempo RTT (ms)	Throughput (img/s)	RAM Pico (MB)
HTTP/REST	JSON	≈ 16500	3,636	2600
gRPC	Protobuf	≈ 1037	57,850	384
ZeroMQ	Protobuf	≈ 672	89,285	250
ZeroMQ	MsgPack	≈ 540	111,087	209

Cuadro 4.1: Resumen de resultados de transmisión para 60,000 imágenes.

4.6 Resultados: Impacto de la Serialización

La transición de JSON a binario (Protobuf) y el refinamiento final con MessagePack (*schema-less*).

4.7 Conclusiones de la Fase 1

Explica cómo "La tríada ZeroMQ + MessagePack + Zero-Copy se establece como la arquitectura ganadora" dentro de Systemd.

Conclusión de esta fase: La elección de ZeroMQ + MessagePack como pila de red ganadora para integrar en Systemd.

Capítulo 5

Fase 2: Despliegue e Inferencia de Modelos de IA

Este capítulo evoluciona el envío tonto de imágenes a una tarea computacional real.

5.1 Entrenamiento del Modelo en el Master

Explica cómo el nodo Master utiliza PyTorch para entrenar un modelo clasificador convolucional (CNN) sencillo sobre el dataset MNIST. Detalla las métricas del entrenamiento (Epochs, Loss, Accuracy final).

El nodo Master asume la carga computacional de entrenamiento y serializa el modelo (‘.pth’).

5.2 Distribución del Modelo y Datos

Explica cómo se envía el modelo entrenado (ej. serializado con ‘torch.save’ en formato ‘.pt’ o ‘.pth’) hacia los workers utilizando la arquitectura de red ganadora de la Fase 1 (ZeroMQ). Seguidamente, se envían las particiones de imágenes.

Envío del modelo y las particiones mediante la arquitectura de red optimizada en la Fase 1.

5.3 Ejecución de Inferencia en el Edge (Workers)

Los nodos hijos reciben el modelo, lo cargan en memoria (‘torch.load’), y para cada imagen recibida realizan una inferencia (predicción del número escrito a mano).

Los Workers cargan el modelo en memoria y evalúan cada imagen, devolviendo las predicciones agregadas al Master.

5.4 Métricas de Procesamiento (T_{proc})

- Tiempo medio de inferencia por lote.
- Consumo de CPU de los workers durante la carga computacional (Tensores IA).
- Fiabilidad: Exactitud de los números predichos y devueltos al máster.

Evaluación del incremento de estrés en CPU y memoria (T_{proc}) al añadir el overhead matemático de los tensores.

Capítulo 6

Fase 3: Migración a Kubernetes (K3s) y Análisis de Trade-offs

Aquí culmina el propósito del TFG: la comparación directa de orquestadores.

6.1 Migración de la Infraestructura a K3s

Explica la configuración del cluster Kubernetes.

- Master asume el rol de Control Plane.
- Workers asumen roles de Kubelets.
- Traducción de los servicios ‘Systemd Quadlets’ a manifiestos de Kubernetes (‘Deployments’, ‘Services’, ‘Pods’).

Adaptación de los servicios a la semántica de *Deployments* y *Services* de K8s.

6.2 Ejecución de Cargas de Trabajo (Fase 1 y 2 en K8s)

Se repite el envío masivo de imágenes y la inferencia de IA, pero ahora a través de la red virtual de Kubernetes (ej. Flannel o Calico CNI) y Kube-proxy.

Ejecución de la Tarea 2 a través de la interfaz de red de contenedores (CNI) de Kubernetes.

6.3 Comparativa Directa: Systemd vs Kubernetes

Aquí comparas los resultados de Systemd frente a K8s usando las métricas definidas:

- **Plano de Gestión (Reposo):** Consumo de RAM y CPU de Systemd+Podman inactivo frente al Kubelet+Kube-proxy+Etcd inactivos en el Edge. Comparativa del *overhead* en reposo del Kubelet frente al demonio de Systemd.

- **Plano de Datos (Red):** Cuánta latencia extra añade el CNI de K8s a gRPC y ZeroMQ frente a la comunicación nativa P2P de Podman. Impacto del enrutamiento de Kube-proxy frente a la red nativa punto a punto.
- **Complejidad Operativa:** Facilidad de despliegue usando Ansible estático vs. mantenimiento del estado en K8s.

Capítulo 7

Conclusiones y Trabajo Futuro

7.1 Conclusiones Finales

Resumen general. Responde a la pregunta del TFG: ¿Vale la pena el peso de Kubernetes en el Edge, o es Systemd una alternativa viable? Destaca los números clave: 1) La inviabilidad de REST/JSON. 2) La eficiencia extrema de ZeroMQ+MsgPack en Podman. 3) El *overhead* demostrado por Kubernetes frente al despliegue desnudo.

Resumen de los hitos alcanzados y la viabilidad técnica demostrada a lo largo de los capítulos.

7.2 Respuesta al Dilema: ¿Systemd o Kubernetes para el Edge?

Veredicto final apoyado en las métricas extraídas en la Fase 3. Cuándo merece la pena asumir el coste de Kubernetes y cuándo es preferible la simplicidad de Systemd.

7.3 Trabajo Futuro

Sugerencias para ampliar el trabajo:

- Probar aceleradores hardware en los Workers (GPU/NPU).
- Usar un framework de IA específico para Edge como TensorFlow Lite o ONNX Runtime.
- Implementar mecanismos de tolerancia a fallos automáticos en el lado de Systemd.

Bibliografía

- [1] Red Hat, *Podman Documentation*, [Online]. Disponible en: <https://podman.io/docs/>
- [2] Freedesktop.org, *systemd.service — Service unit configuration*, [Online].
- [3] Cloud Native Computing Foundation (CNCF), *gRPC: A high performance, open source universal RPC framework*, [Online].
- [4] P. Hintjens, *ZeroMQ: Messaging for Many Applications*, O'Reilly Media, 2013.
- [5] Rancher Labs (SUSE), *K3s Lightweight Kubernetes*, [Online]. Disponible en: <https://k3s.io/>
- [6] Paszke, A. et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.